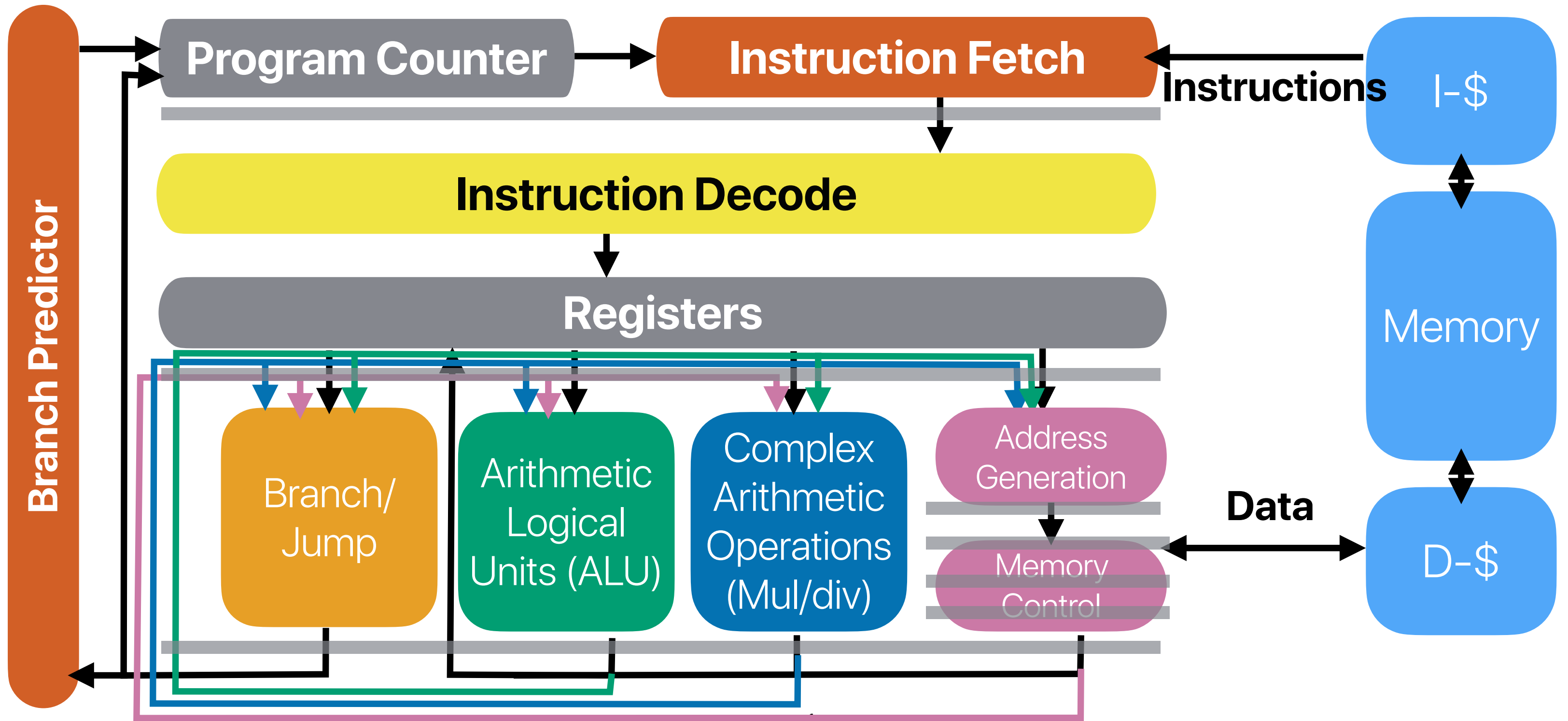
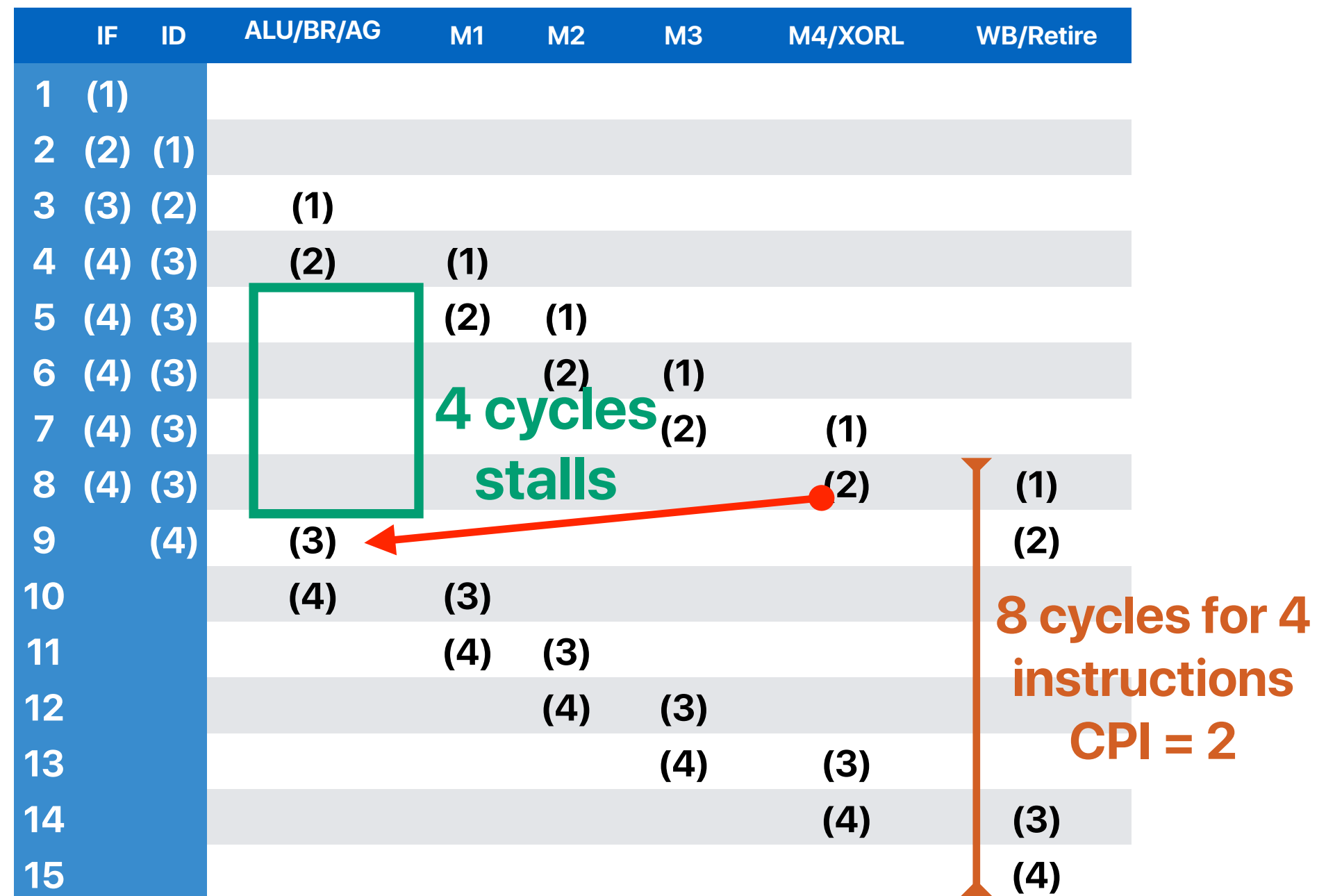


Recap: Data "forwarding"



The effect of data forwarding

- ① `movl (%rdi), %eax`
- ② `movl (%rsi), %edx`
- ③ `movl %edx, (%rdi)`
- ④ `movl %eax, (%rsi)`



Missing opportunities

```
for(i = 0; i < count; i++) {  
    int64_t temp = a[i];  
    a[i] = b[i];  
    b[i] = temp;  
}
```

Compiler can only do this when it's 100% for sure count is
always an even number! — loop unrolling

Compilers are limited by the number of registers available to the software!

```
:  
movq    (%rcx,%rax), %r8  
movq    (%rdi,%rax), %rsi  
addq    $8, %rax  
movq    %r8, -8(%rdi,%rax)  
movq    %rsi, -8(%rcx,%rax)  
cmpq    %r9, %rax  
jne     .L9  
movq    (%rcx,%rax), %r8  
movq    (%rdi,%rax), %rsi  
addq    $8, %rax  
movq    %r8, -8(%rdi,%rax)  
movq    %rsi, -8(%rcx,%rax)  
cmpq    %r9, %rax  
jne     .L9  
.L9:  
movq    (%rcx,%rax), %r8  
movq    (%rdi,%rax), %rsi  
addq    $8, %rax  
movq    %r8, -8(%rdi,%rax)  
movq    %rsi, -8(%rcx,%rax)  
cmpq    %r9, %rax  
jne     .L9
```



	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5	(5)	(4)	(3)	(2)	(1)			
6	(6)	(5)	(4)	(3)	(2)	(1)		
7	(7)	(6)	(5)	(4)	(3)	(2)	(1)	
8	(8)	(7)	(6)	(5)	(4)	(3)	(2)	(1)
9	(9)	(8)	(7)	(6)	(5)	(4)	(3)	(2)
10	(10)	(9)	(8)	(7)	(6)	(5)	(4)	(3)
11	(11)	(10)	(9)	(8)	(7)	(6)	(5)	(4)
12	(12)	(11)	(10)	(9)	(8)	(7)	(6)	(5)
13	(13)	(12)	(11)	(10)	(9)	(8)	(7)	(6)
14	(14)	(13)	(12)	(11)	(10)	(9)	(8)	(7)
15	(15)	(14)	(13)	(12)	(11)	(10)	(9)	(8)
16	(16)	(15)	(14)	(13)	(12)	(11)	(10)	(9)
17	(17)	(16)	(15)	(14)	(13)	(12)	(11)	(10)
18	(18)	(17)	(16)	(15)	(14)	(13)	(12)	(11)
19	(19)	(18)	(17)	(16)	(15)	(14)	(13)	(12)
20	(20)	(19)	(18)	(17)	(16)	(15)	(14)	(13)
21	(21)	(20)	(19)	(18)	(17)	(16)	(15)	(14)
22	(22)	(21)	(20)	(19)	(18)	(17)	(16)	(15)
23	(23)	(22)	(21)	(20)	(19)	(18)	(17)	(16)

7 cycles for 7
instructions

CPI = 1

What do you need to execution an instruction?

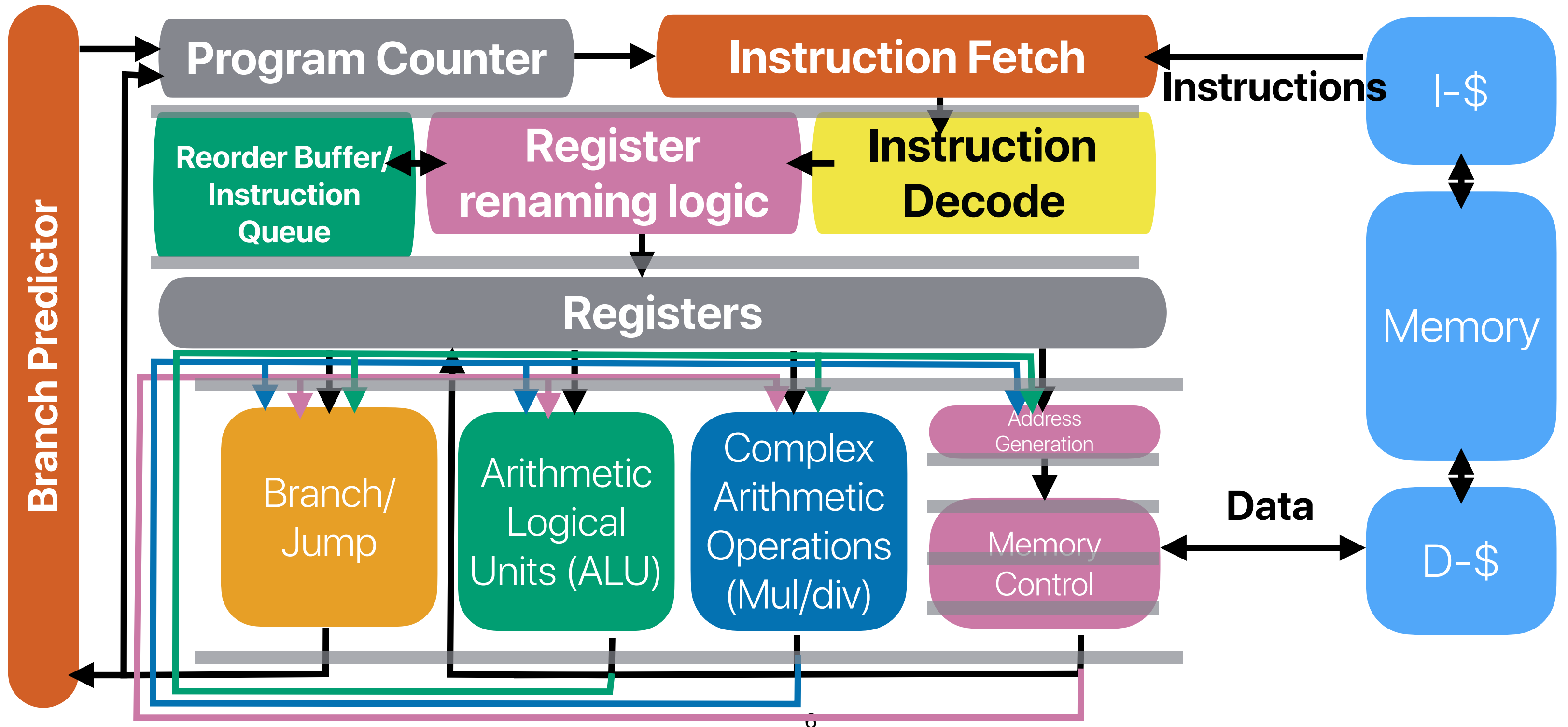
- Whenever the instruction is decoded — put decoded instruction somewhere
- Whenever the inputs are ready — **all data dependencies are resolved**
- Whenever the target functional unit is available

False dependencies

- We are still limited by **false dependencies**
- They are not “true” dependencies because they don’t have an arrow in data dependency graph
 - WAR (Write After Read): a later instruction overwrites the source of an earlier one
 - 5 and 1, 5 and 2, 12 and 8, 12 and 9
 - WAW (Write After Write): a later instruction overwrites the output of an earlier one



Register renaming + OoO + RoB



Only 1 of them can have a instruction at the same cycle

Register renaming

- ① movq (%rdi,%rax), %rsi → P1
- ② movq (%rcx,%rax), %r8 → P2
- ③ movq %r8 (P1), (%rdi,%rax)
- ④ movq %rsi(P2), (%rcx,%rax)
- ⑤ addq \$8, %rax → P3
- ⑥ cmpq %r9, %rax (P3)
- ⑦ jne .L9
- ⑧ movq (%rdi,%rax), %rsi → P4
- ⑨ movq (%rcx,%rax), %r8 → P5
- ⑩ movq %r8 (P4), (%rdi,%rax)
- ⑪ movq %rsi(P11), (%rcx,%rax)
- ⑫ addq \$8, %rax → P6
- ⑬ cmpq %r9, %rax (P6)
- ⑭ jne .L9
- ⑮ movq (%rdi,%rax), %rsi
- ⑯ movq (%rcx,%rax), %r8
- ⑰ movq %r8, (%rdi,%rax)
- ⑱ movq %rsi, (%rcx,%rax)
- ⑲ addq \$8, %rax
- ⑳ cmpq %r9, %rax
- ㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			(3)(4)(5)(6)(7)
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				(8)(12)(13)(14)
19	(19)	(18)	(15)(16)(17)	(10)	(11)							(9)(12)(13)(14)
20	(20)	(19)	(16)(17)(18)	(15)	(10)	(11)						(12)(13)(14)
21	(21)	(20)	(17)(18)(19)	(16)	(15)	(10)	(11)					(12)(13)(14)
22		(21)	(17)(18)(20)		(16)	(15)	(10)	(11)	(19)			(12)(13)(14)
23			(17)(20)(21)	(18)		(16)	(15)	(10)				(11)(12)(13)(14)(19)
24			(20)(21)	(17)	(18)		(16)	(15)				(10)(11)(12)(13)(14)(19)
25			(21)		(17)	(18)		(16)	(20)			(15)(19)

7 cycles for 7 instructions
CPI = 1

Takeaways: data hazards

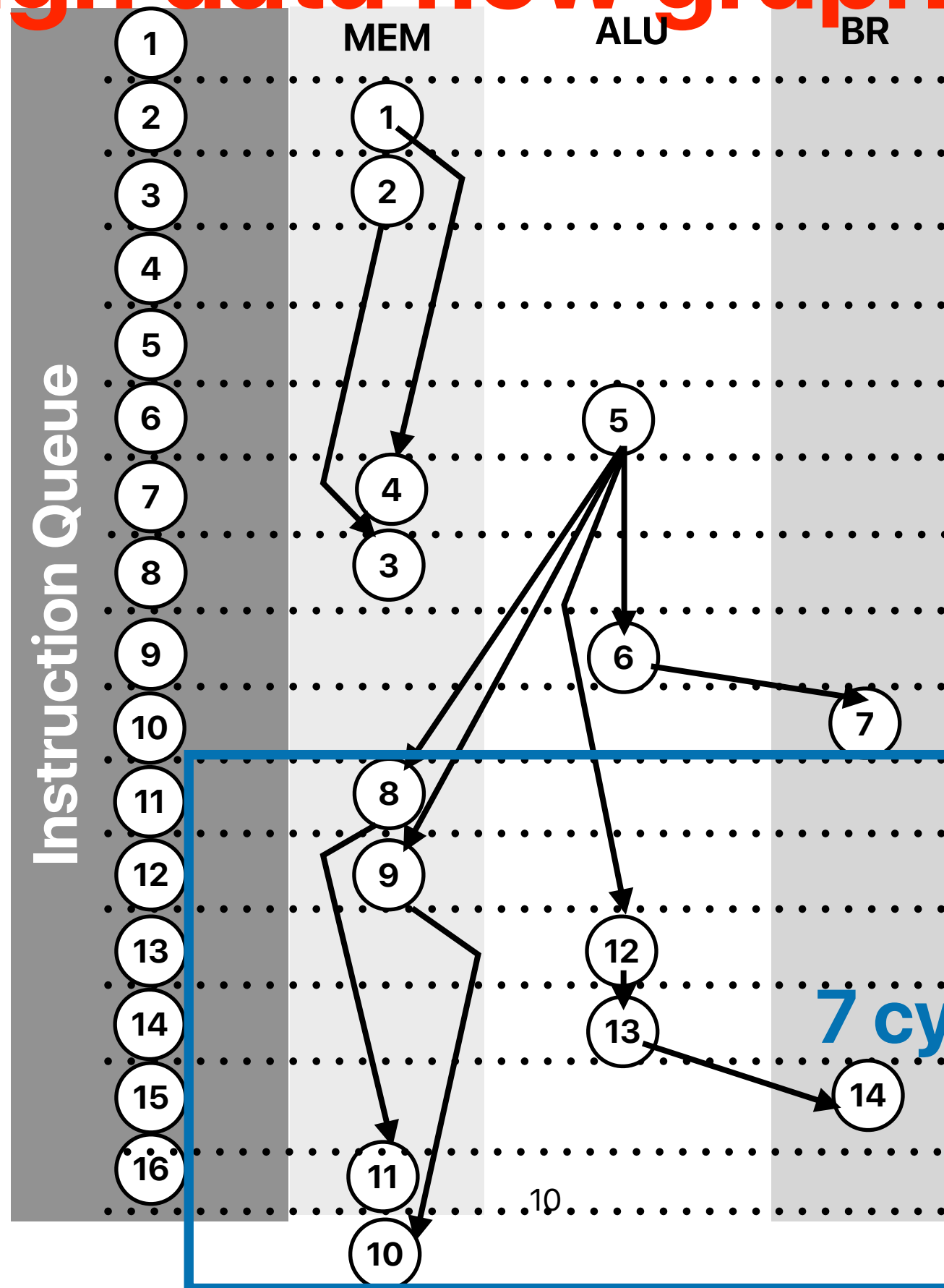
- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent
- False dependencies limits the freedom of out-of-order execution
- Register renaming + Speculative execution enables more efficient execution by dynamically scheduling instructions whenever their data dependencies are resolved

The mechanism of OoO: Register renaming + speculative execution (cont.)

- K. C. Yeager, "The MIPS R10000 superscalar microprocessor," in IEEE Micro, vol. 16, no. 2, pp. 28-41, April 1996.

Through data flow graph analysis

5 cycles for memory ops
1 cycle for ALU/branch ops
1-issue pipeline



7 cycles every iteration

$$\text{CPI} = \frac{7}{7} = 1!$$

```
① movq (%rdi,%rax), %rsi
② movq (%rcx,%rax), %r8
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq $8, %rax
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi
⑨ movq (%rcx,%rax), %r8
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq $8, %rax
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq $8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9
```

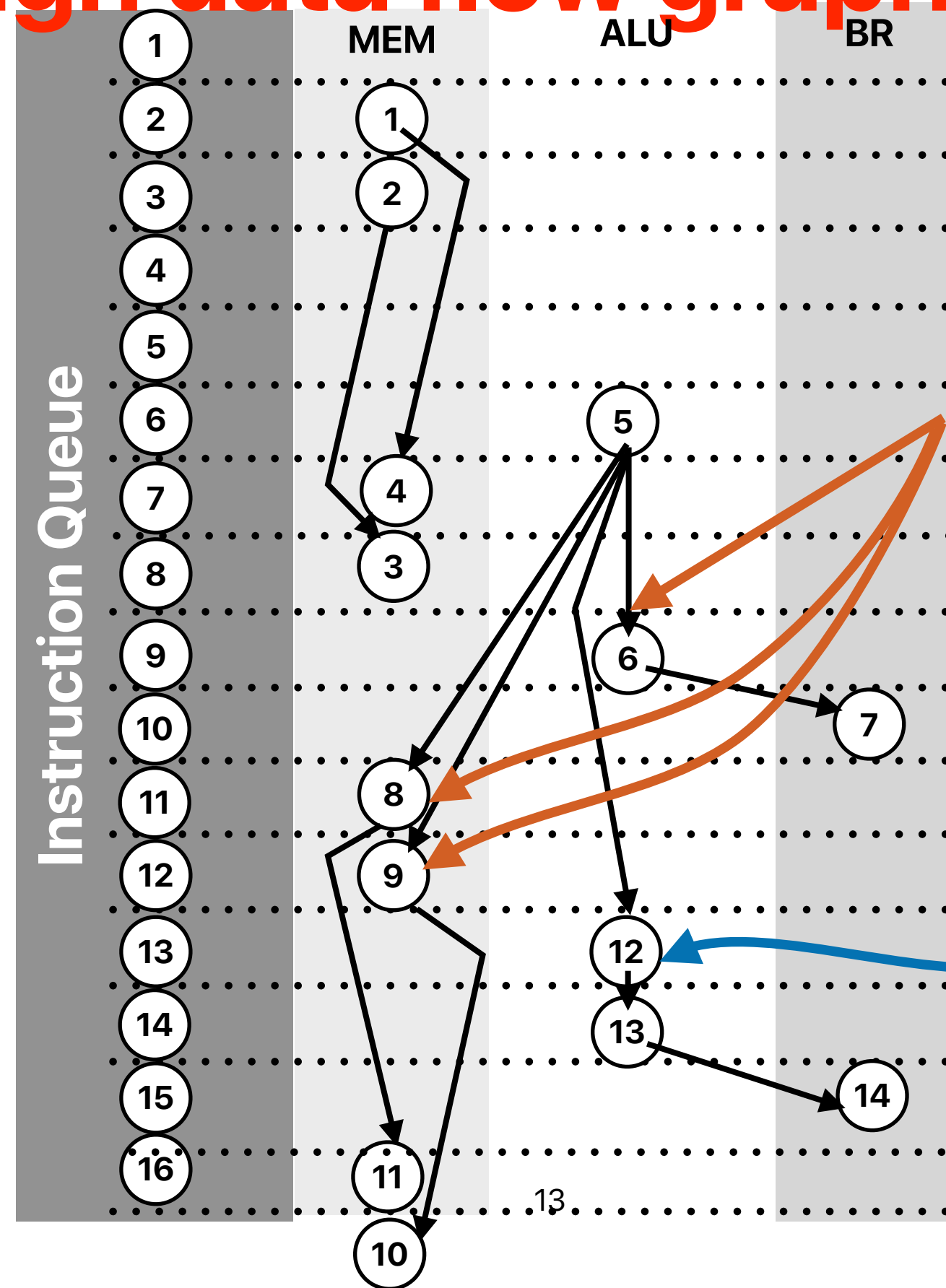
Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent
- False dependencies limits the freedom of out-of-order execution
- Register renaming + Speculative execution enables more efficient execution by dynamically scheduling instructions whenever their data dependencies are resolved

If $CPI == 1$ the limitation?

Through data flow graph analysis

```
① movq (%rdi,%rax), %rsi
② movq (%rcx,%rax), %r8
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq $8, %rax
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi
⑨ movq (%rcx,%rax), %r8
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq $8, %rax
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq $8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9
```



5 cycles for memory ops
1 cycle for ALU/branch ops
1-issue pipeline

We cannot issue them earlier simply because structural hazards!

We could have this executed earlier if it's in the queue earlier

Super Scalar

Superscalar

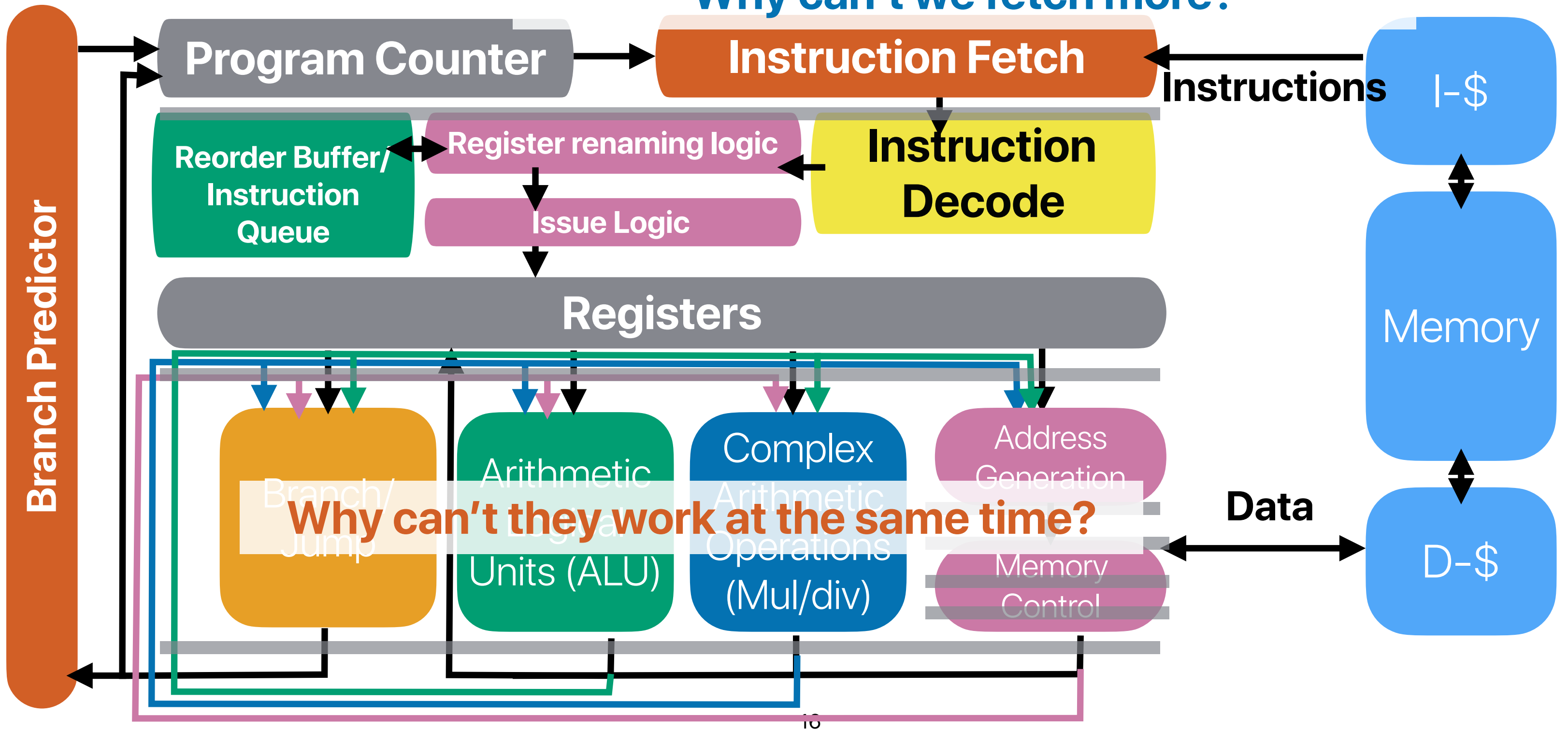
- Since we have many functional units now, we should fetch/decode more instructions each cycle so that we can have more instructions to issue!
- Super-scalar: fetch/decode/issue more than one instruction each cycle
 - **Fetch width:** how many instructions can the processor fetch/decode each cycle
 - **Issue width:** how many instructions can the processor issue each cycle
- The theoretical CPI should now be

1

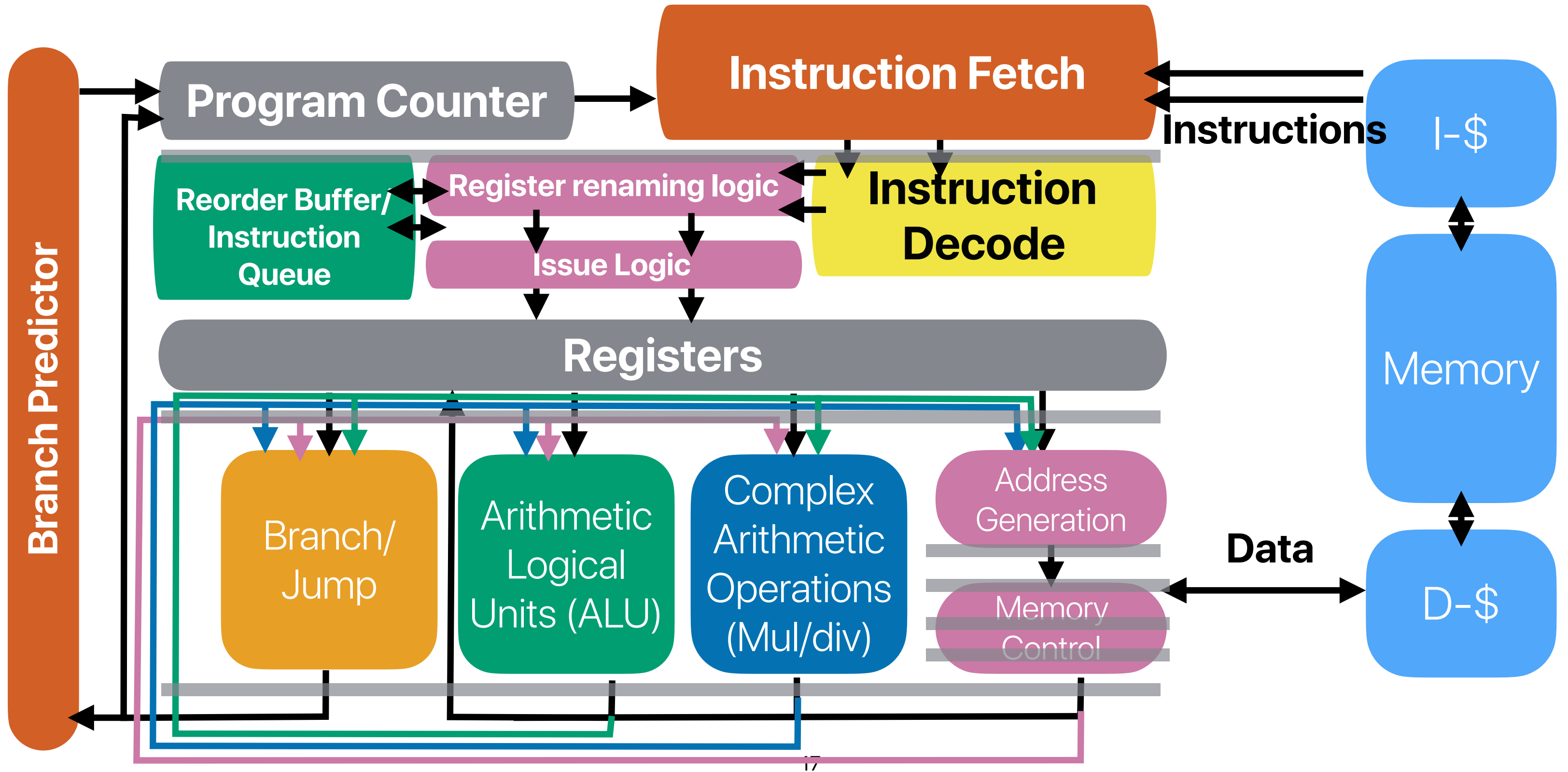
$\min(\text{issue width}, \text{fetch width}, \text{decode width})$

Register renaming + OoO + RoB

Why can't we fetch more?



Register renaming + SuperScalar



2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4												
5												
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5												
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)	(2)	(1)							
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)					(5)
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												
21												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9) (10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11) (12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												
21												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9) (10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11) (12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13) (14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												
21												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20)	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20)	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20)	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20)	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
16												
17												
18												
19												
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20)	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(12)(13)(14)(15)(19)(20)(21)
17												
18												
19												
20												
21												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20)	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
17					(17)		(16)	(10)				(11)(12)(13)(14)(15)(19)(20)(21)
18												
19												
20												
21												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20)	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(12)(13)(14)(15)(19)(20)(21)
17					(17)		(16)	(10)				(11)(12)(13)(14)(15)(19)(20)(21)
18						(17)		(16)				(10)(11)(12)(13)(14)(15)(19)(20)(21)
19												
20												
21												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20)	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(12)(13)(14)(15)(19)(20)(21)
17					(17)		(16)	(10)				(11)(12)(13)(14)(15)(19)(20)(21)
18						(17)		(16)				(10)(11)(12)(13)(14)(15)(19)(20)(21)
19				(18)			(17)					(16)(19)(20)(21)
20												

2-issue SS + Register renaming + OoO

2 issue: "2" of them can have a instruction at the same cycle

① movq (%rdi,%rax), %rsi → P1
② movq (%rcx,%rax), %r8 → P2
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq \$8, %rax → P3
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi → P4
⑨ movq (%rcx,%rax), %r8 → P5
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq \$8, %rax → P6
⑬ cmpq %r9, %rax
⑭ jne .L9
⑮ movq (%rdi,%rax), %rsi
⑯ movq (%rcx,%rax), %r8
⑰ movq %r8, (%rdi,%rax)
⑱ movq %rsi, (%rcx,%rax)
⑲ addq \$8, %rax
⑳ cmpq %r9, %rax
㉑ jne .L9

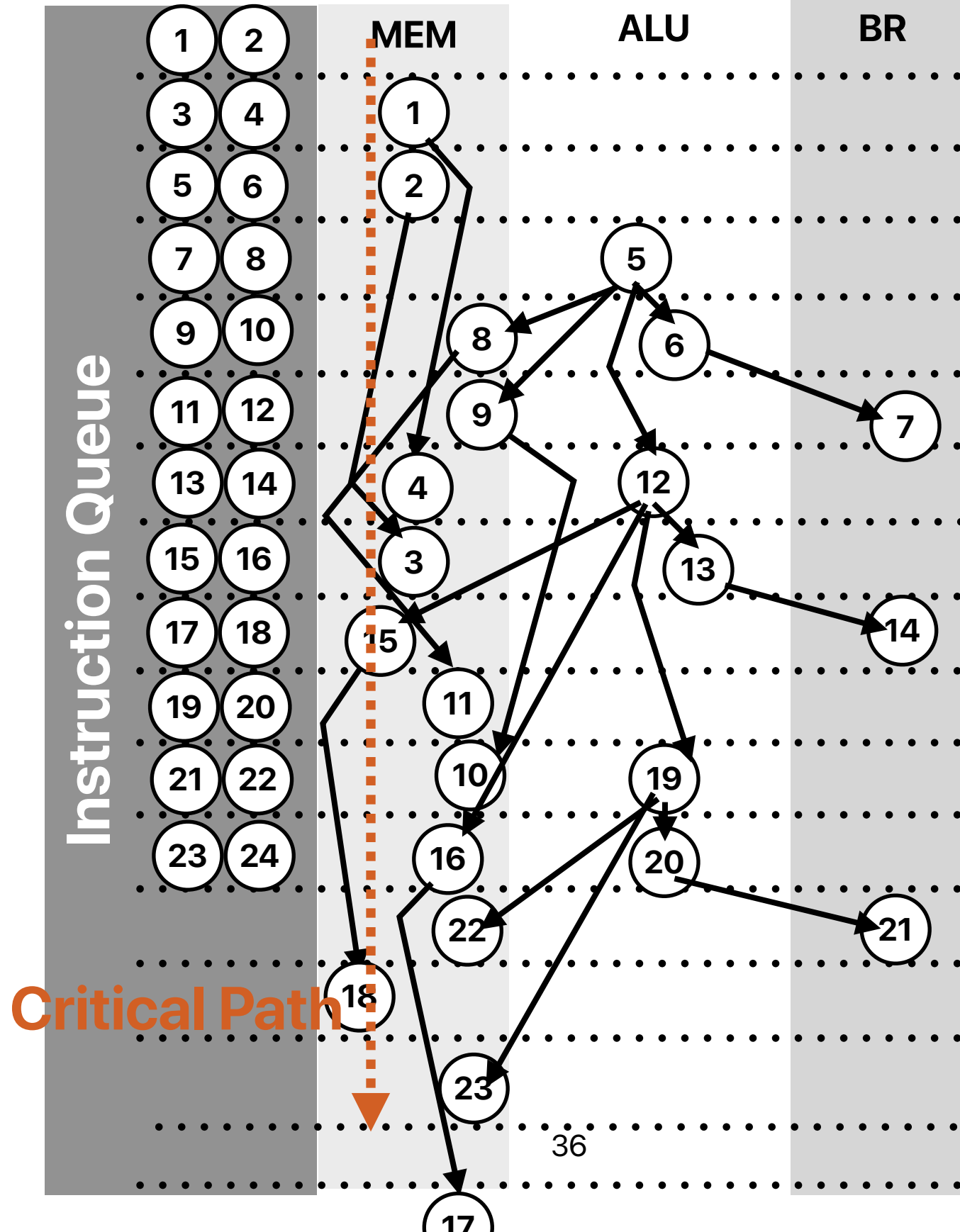
	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1) (5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2) (5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20)	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(2)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(12)(13)(14)(15)(19)(20)(21)
17					(17)		(16)	(10)				(11)(12)(13)(14)(15)(19)(20)(21)
18						(17)		(16)				(10)(11)(12)(13)(14)(15)(19)(20)(21)
19				(18)			(17)					(10) (19)(20)(21)
20					(18)			(17)				(19)(20)(21)

Through data flow graph analysis

```

①  movq (%rdi,%rax), %rsi
②  movq (%rcx,%rax), %r8
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
㉒  movq (%rdi,%rax), %rsi
㉓  movq (%rcx,%rax), %r8
㉔  movq %r8, (%rdi,%rax)
㉕  movq %rsi, (%rcx,%rax)
㉖  addq $8, %rax
㉗  cmpq %r9, %rax

```



5 cycles for memory ops
1 cycle for ALU/branch ops
2-issue pipeline

12 cycles for every 11 memory instructions

**If we have 11 loops, it
will have 44 memory
instructions, 77
instructions in total
and take 48 cycles**

CPI: $\frac{48}{77} = 0.62$

Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent
- False dependencies limits the freedom of out-of-order execution
- Register renaming + Speculative execution enables more efficient execution by dynamically scheduling instructions whenever their data dependencies are resolved
- Super scalar further improves the utilization of hardware and throughput

The pipelines of Modern Processors

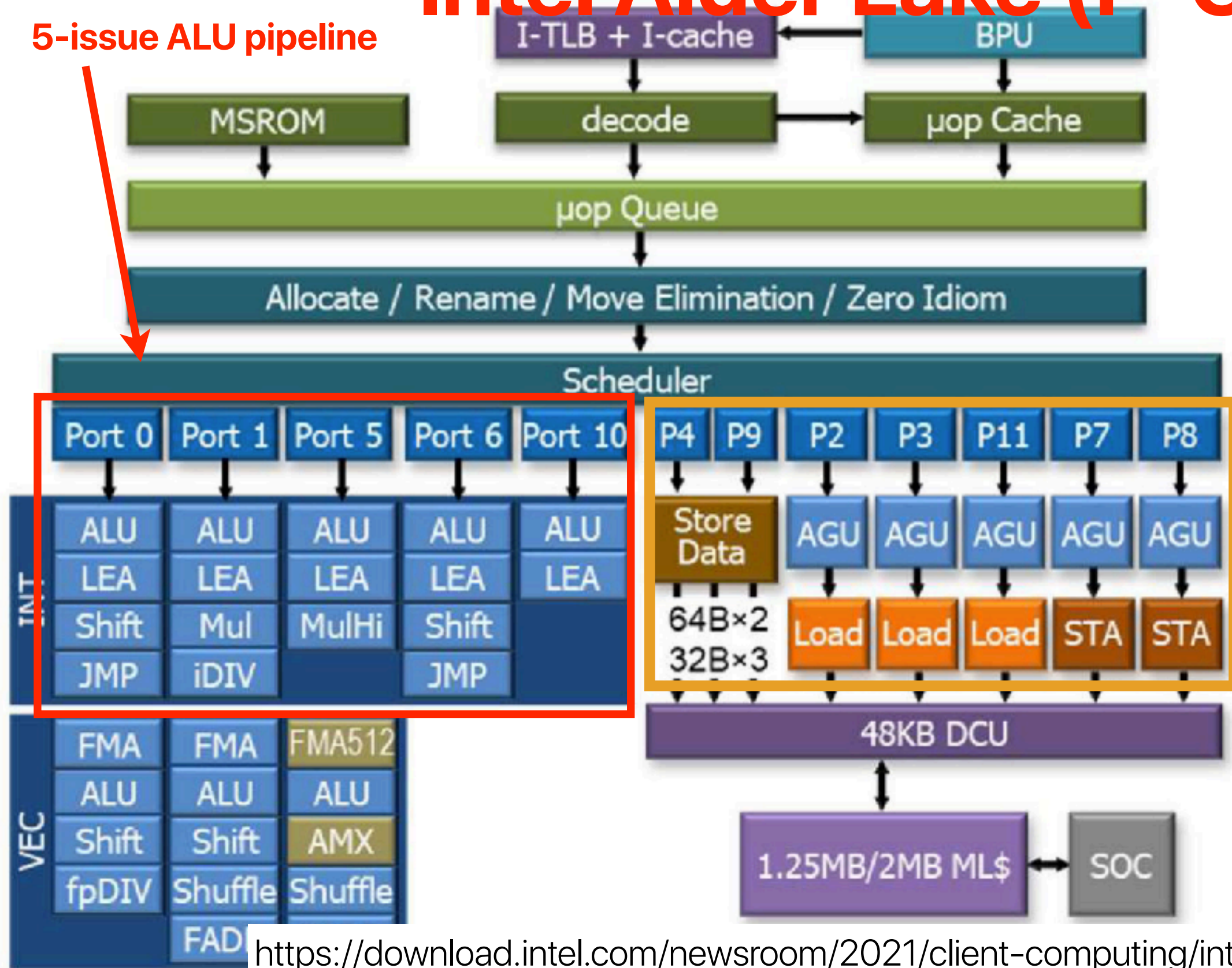
Intel Alder Lake (P-Core)

$$MinCPI = \frac{1}{12}$$

$$MinINTInst.CPI = \frac{1}{5}$$

$$MinMEMInst.CPI = \frac{1}{7}$$

$$MinBRInst.CPI = \frac{1}{2}$$



7-issue memory pipeline

Intel Skylake — your assignment

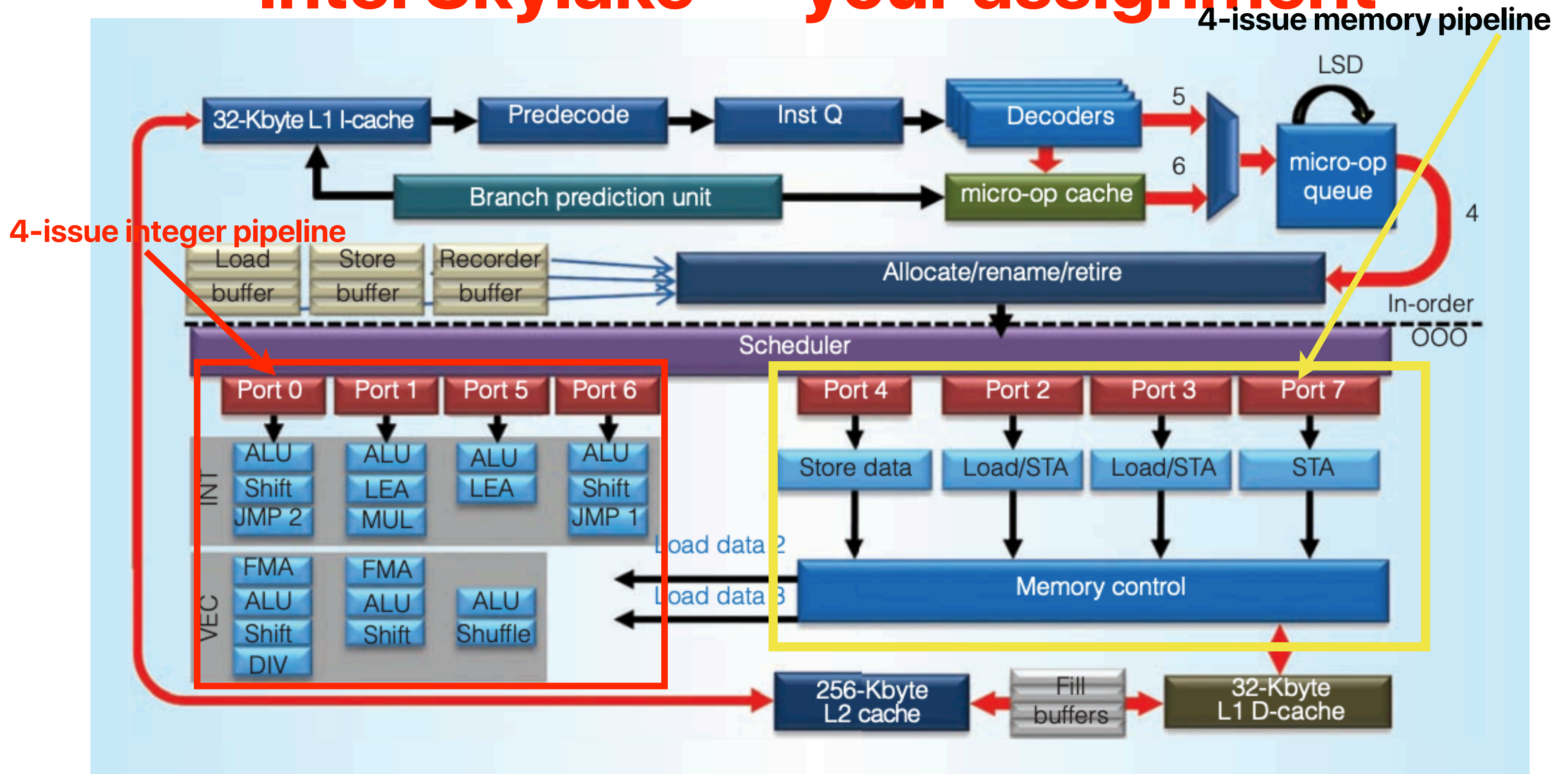
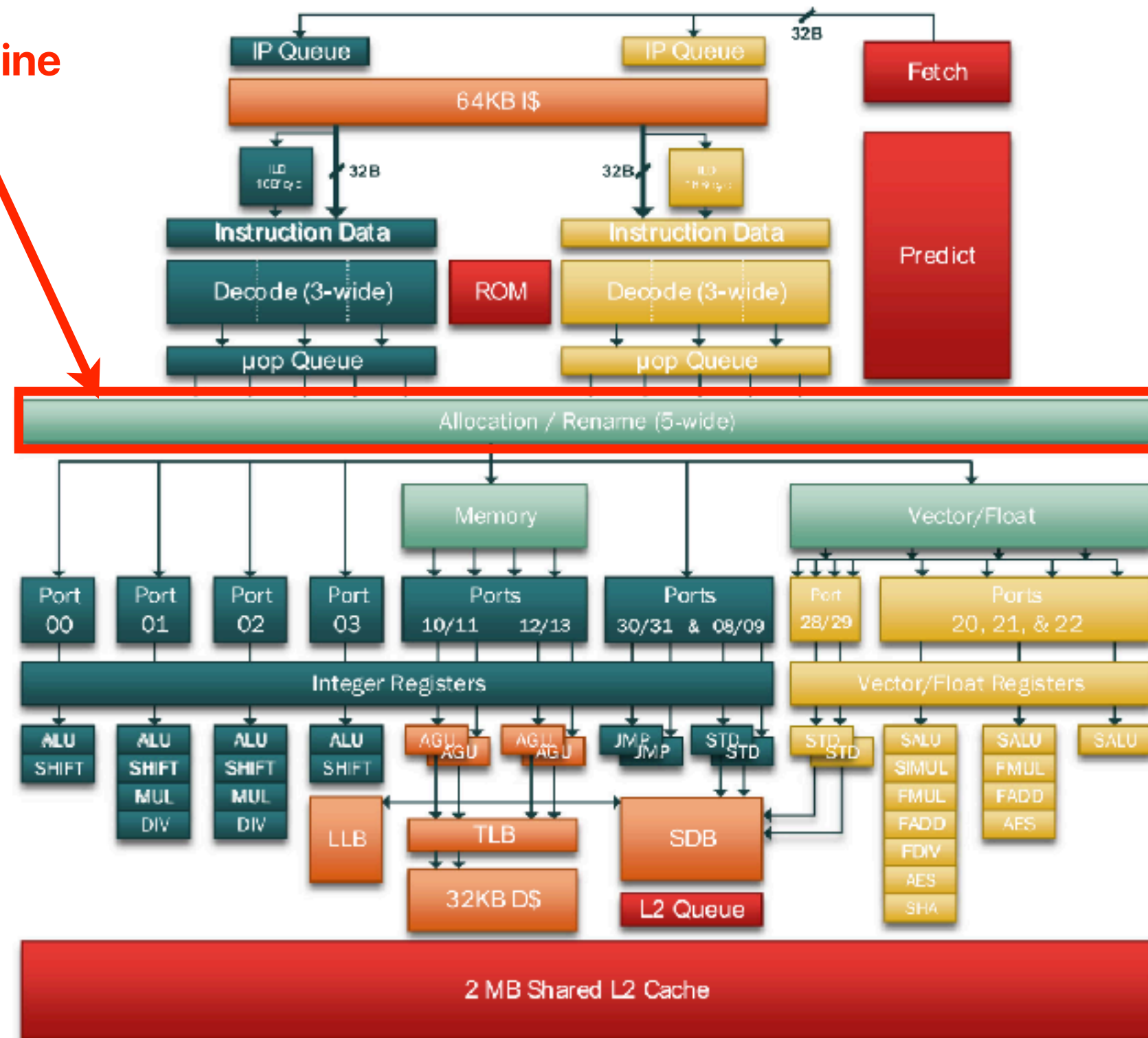


Figure 4. Skylake core block diagram.

Intel Alder Lake (E-Core)

5-issue pipeline



AMD Zen 3 (RyZen 5000 Series)

3-issue memory pipeline

4-issue integer pipeline + 1 additional branch

$$MinCPI = \frac{1}{8}$$

$$MinINTInst.CPI = \frac{1}{4}$$

$$MinMEMInst.CPI = \frac{1}{3}$$

$$MinBRInst.CPI = \frac{1}{2}$$

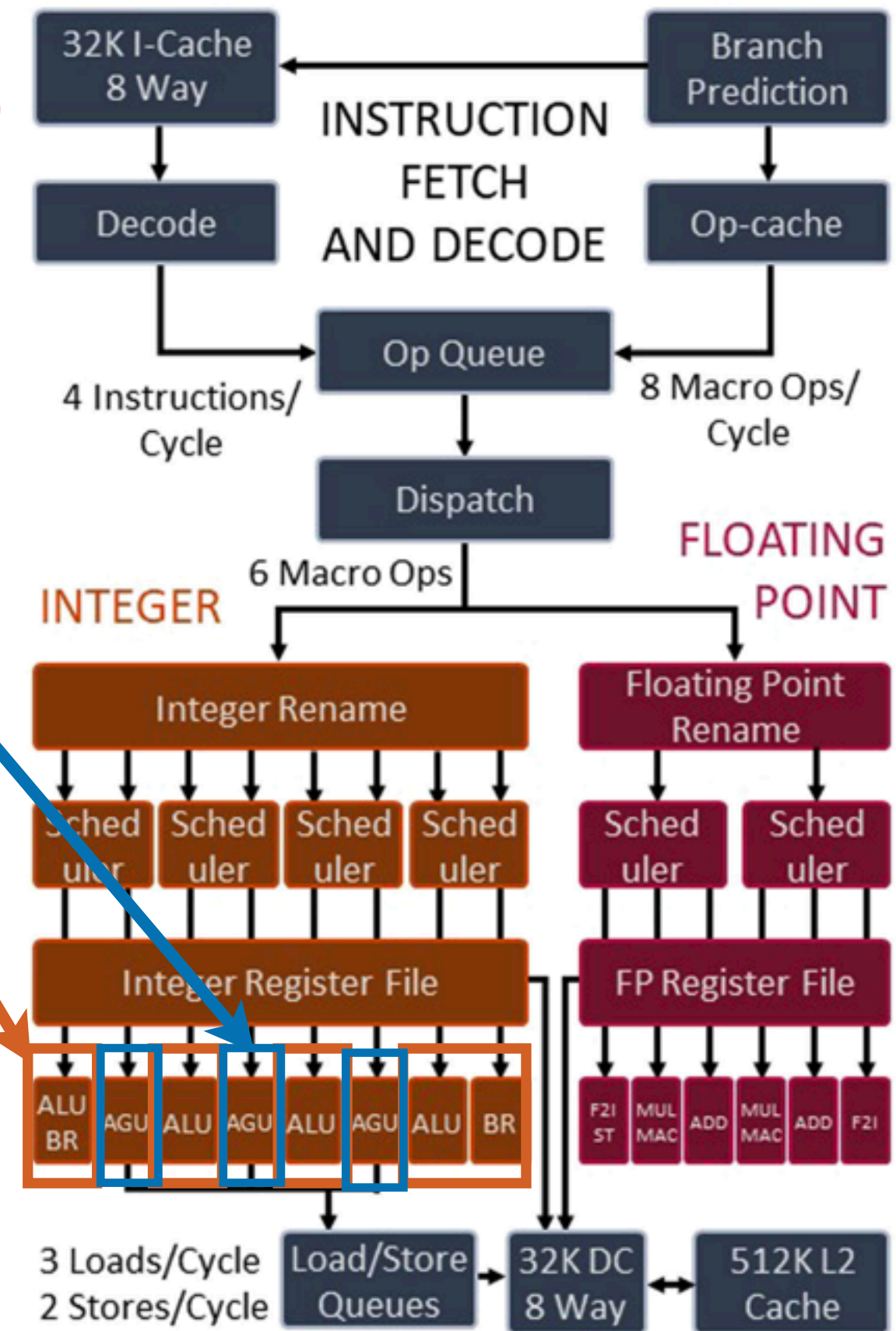


FIGURE 1. "Zen 3" block diagram.

**Refresh our minds! What are the
characteristics of modern
processors?**

Summary: Characteristics of modern processor architectures

- Multiple-issue pipelines with multiple functional units available
 - Multiple ALUs
 - Multiple Load/store units
 - Dynamic OoO scheduling to reorder instructions whenever possible
- Cache — very high hit rate **if your code has good locality**
 - Very matured data/instruction prefetcher
- Branch predictors — very high accuracy **if your code is predictable**
 - Perceptron
 - TAGE

**How to “program” modern
processors efficiently?**



Linked list v.s. arrays

- We can use either a linked list or an array to store a list of data, compare the performance of the array (version A) and linked list (version B) implementations that can achieve the same outcome as below. Assume we have a processor with a reasonably good branch predictor and **unlimited** fetch/issue width and the dataset size is small enough to **fit inside the L1 cache**, please identify the correct statements.

- ① There is very little performance difference between A and B
- ② B will outperform A as A has more branch instructions
- ③ A will outperform B as A has fewer cache misses
- ④ A will outperform B as A has fewer data dependance related stalls
- ⑤ A will outperform B as A has fewer dynamic instructions

- A. 0
B. 1
C. 2
D. 3
E. 4

A

```
for(i=0;i<size;i++)  
{  
    if(node[i].next)  
        number_of_nodes++;  
}
```

B

```
while(node)  
{  
    node = node->next;  
    number_of_nodes++;  
}
```



Linked list v.s. arrays

- We can use either a linked list or an array to store a list of data, compare the performance of the array (version A) and linked list (version B) implementations that can achieve the same outcome as below. Assume we have a processor with a reasonably good branch predictor and **unlimited** fetch/issue width and the dataset size is small enough to **fit inside the L1 cache**, please identify the correct statements.

- ① There is very little performance difference between A and B
- ② B will outperform A as A has more branch instructions
- ③ A will outperform B as A has fewer cache misses
- ④ A will outperform B as A has fewer data dependance related stalls
- ⑤ A will outperform B as A has fewer dynamic instructions

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

A

```
for(i=0;i<size;i++)
{
    if(node[i].next)
        number_of_nodes++;
}
```

B

```
while(node)
{
    node = node->next;
    number_of_nodes++;
}
```

Linked list v.s. arrays

- We can use either a linked list or an array to store a list of data, compare the performance of the array (version A) and linked list (version B) implementations that can achieve the same outcome as below. Assume we have a processor with a reasonably good branch predictor and **unlimited** fetch/issue width and the dataset size is small enough to **fit inside the L1 cache**, please identify the correct statements.

- ① There is very little performance difference between A and B
- ② B will outperform A as A has more branch instructions
- ③ A will outperform B as A has fewer cache misses **about the same**
- ④ A will outperform B as A has fewer data dependance related stalls
- ⑤ A will outperform B as A has fewer dynamic instructions

- A. 0
B. 1
C. 2
D. 3
E. 4

A

```
for(i=0;i<size;i++)  
{  
    if(node[i].next)  
        number_of_nodes++;  
}
```

B

```
while(node)  
{  
    node = node->next;  
    number_of_nodes++;  
}
```

Take a look of their instructions

A

```
for(i=0;i<size;i++)
{
    if(node[i].next)
        number_of_nodes++;
}
```

```
① .L9:    cmpq    $1, 8(%rax)
②          sbb1    $-1, %edx
③          addq    $16, %rax
④          cmpq    %rdi, %rax
⑤          jne     .L9
```

If we have n iterations, we will execute $5n$ instructions.

B

```
while(node)
{
    node = node->next;
    number_of_nodes++;
}
```

```
① .L3:    movq    8(%rdi), %rdi
②          addl    $1, %eax
③          testq   %rdi, %rdi
④          jne     .L3
```

If we have n iterations, we will execute $4n$ instructions.

Linked list v.s. arrays

- We can use either a linked list or an array to store a list of data, compare the performance of the array (version A) and linked list (version B) implementations that can achieve the same outcome as below. Assume we have a processor with a reasonably good branch predictor and **unlimited** fetch/issue width and the dataset size is small enough to **fit inside the L1 cache**, please identify the correct statements.

- ① There is very little performance difference between A and B
- ② B will outperform A as A has more branch instructions
- ③ A will outperform B as A has fewer cache misses **about the same**
- ④ A will outperform B as A has fewer data dependance related stalls
- ⑤ A will outperform B as A has fewer dynamic instructions **more instructions**

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

A

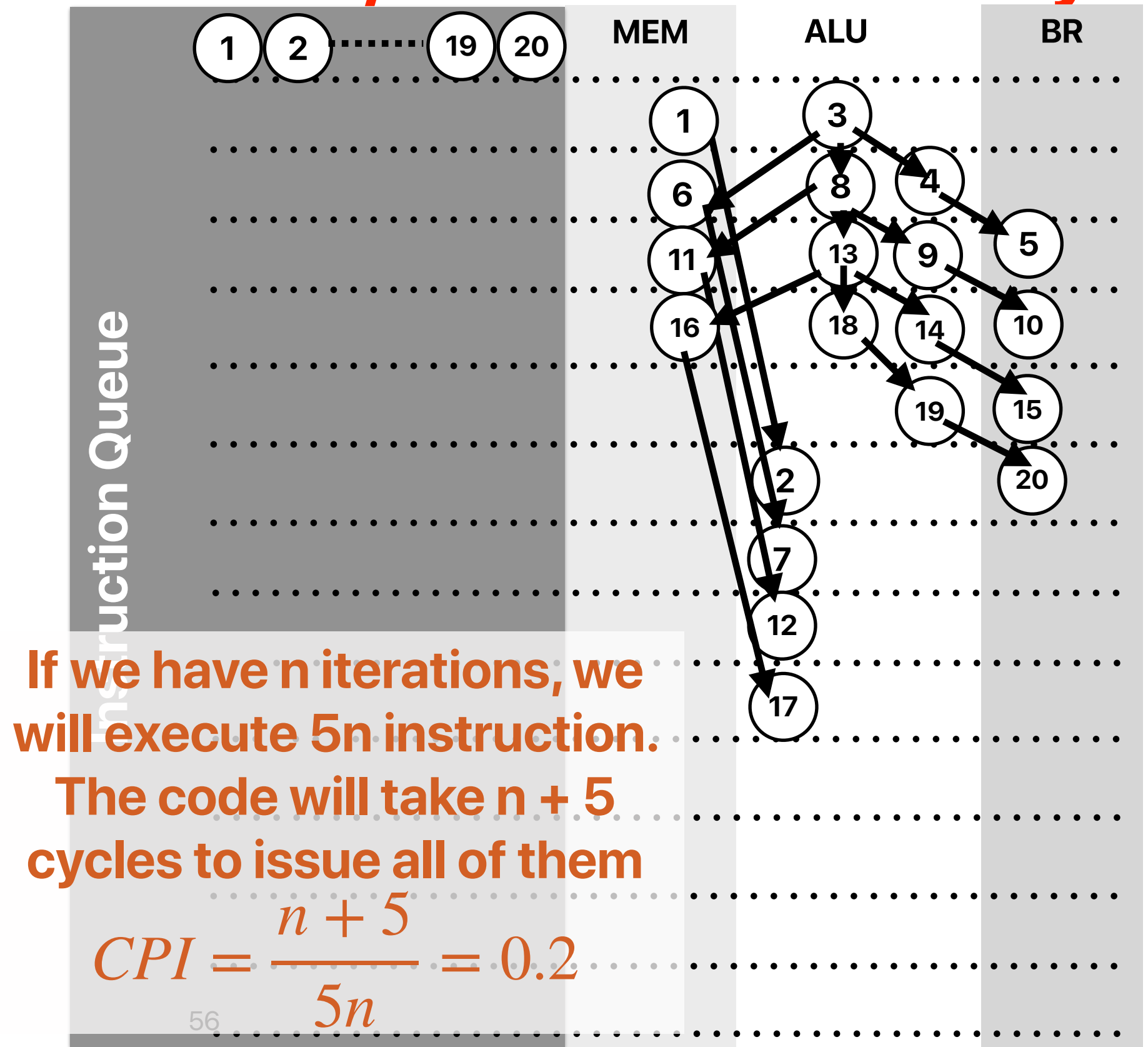
```
for(i=0;i<size;i++)
{
    if(node[i].next)
        number_of_nodes++;
}
```

B

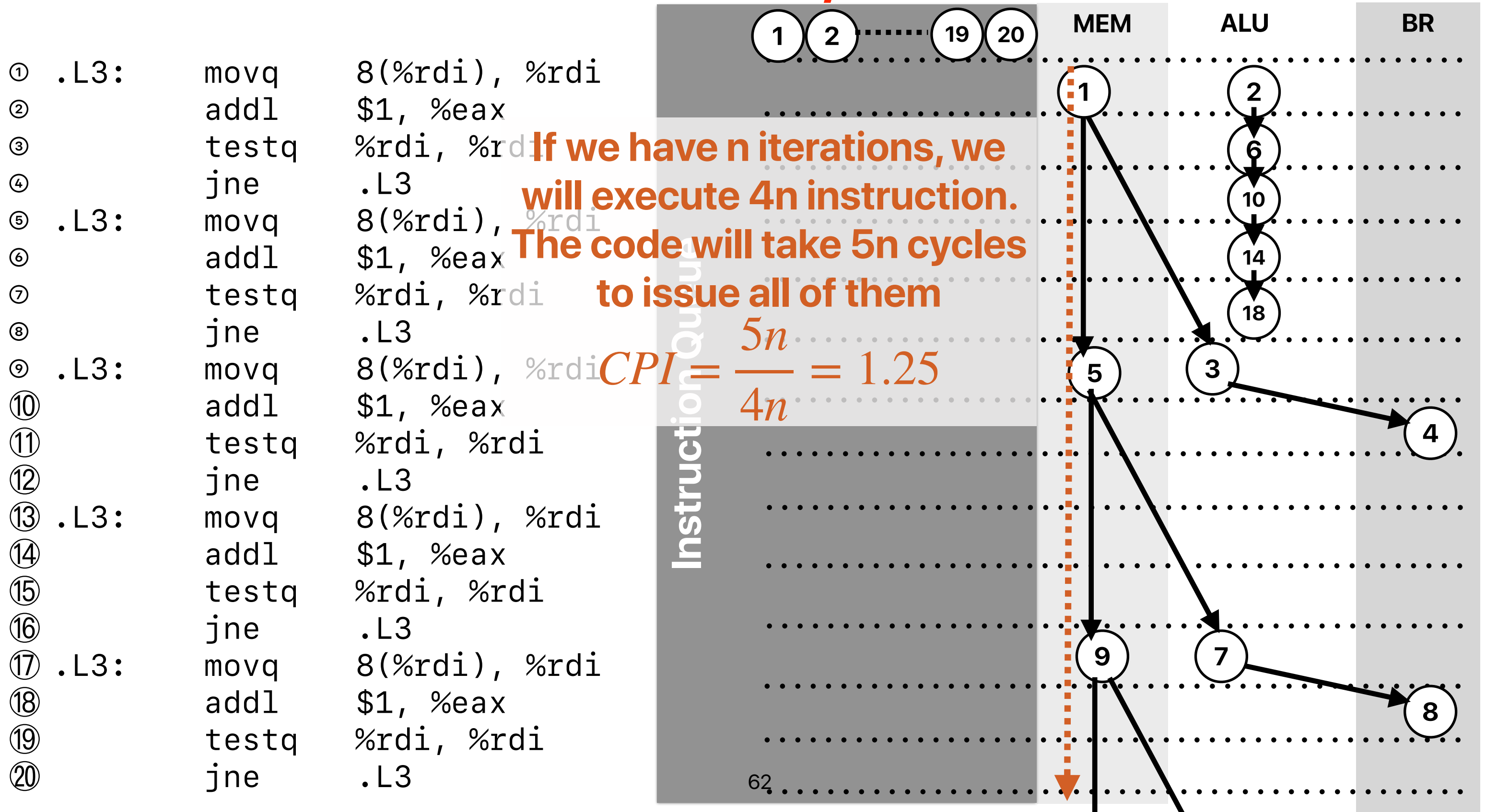
```
while(node)
{
    node = node->next;
    number_of_nodes++;
}
```

What if we have "unlimited" fetch/issue width — "array"

① .L9: cmpq \$1, 8(%rax)
② sbb1 \$-1, %edx
③ addq \$16, %rax
④ cmpq %rdi, %rax
⑤ jne .L9
⑥ .L9: cmpq \$1, 8(%rax)
⑦ sbb1 \$-1, %edx
⑧ addq \$16, %rax
⑨ cmpq %rdi, %rax
⑩ jne .L9
⑪ .L9: cmpq \$1, 8(%rax)
⑫ sbb1 \$-1, %edx
⑬ addq \$16, %rax
⑭ cmpq %rdi, %rax
⑮ jne .L9
⑯ .L9: cmpq \$1, 8(%rax)
⑰ sbb1 \$-1, %edx
⑱ addq \$16, %rax
⑲ cmpq %rdi, %rax
⑳ jne .L9



What if we have "unlimited" fetch/issue width — "linked list"

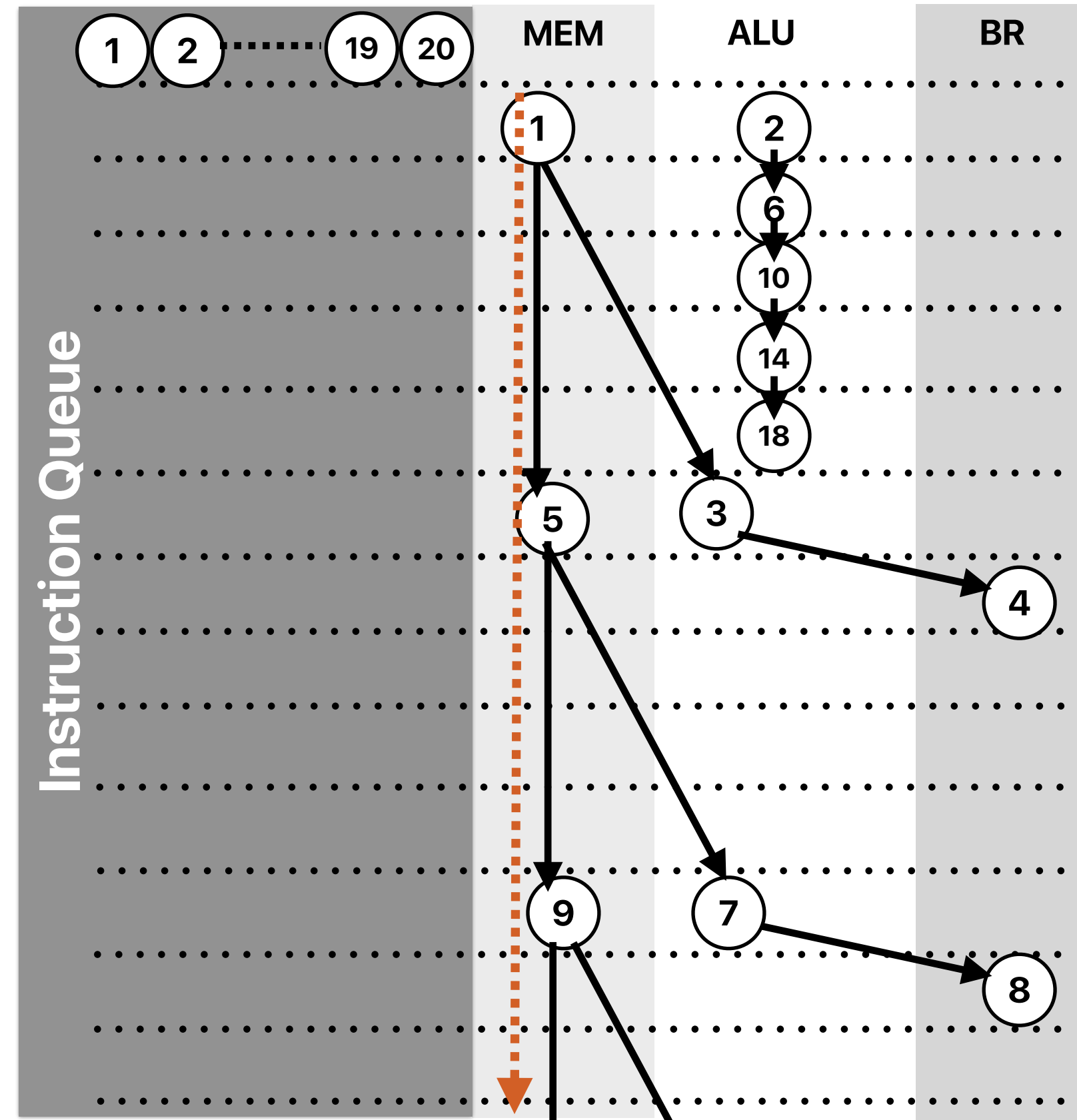


What if we have "unlimited" fetch/issue width — "linked list"

If we cannot improve the performance of executing
`movq 8(%rdi), %rdi`
we cannot improve the execution time.
That's the "critical path"!

```
do {  
    number_of_nodes++;  
    current = current->next;  
} while ( current != NULL );
```

①	.L3:	movq	8(%rdi), %rdi
②		addl	\$1, %eax
③		testq	%rdi, %rdi
④		jne	.L3



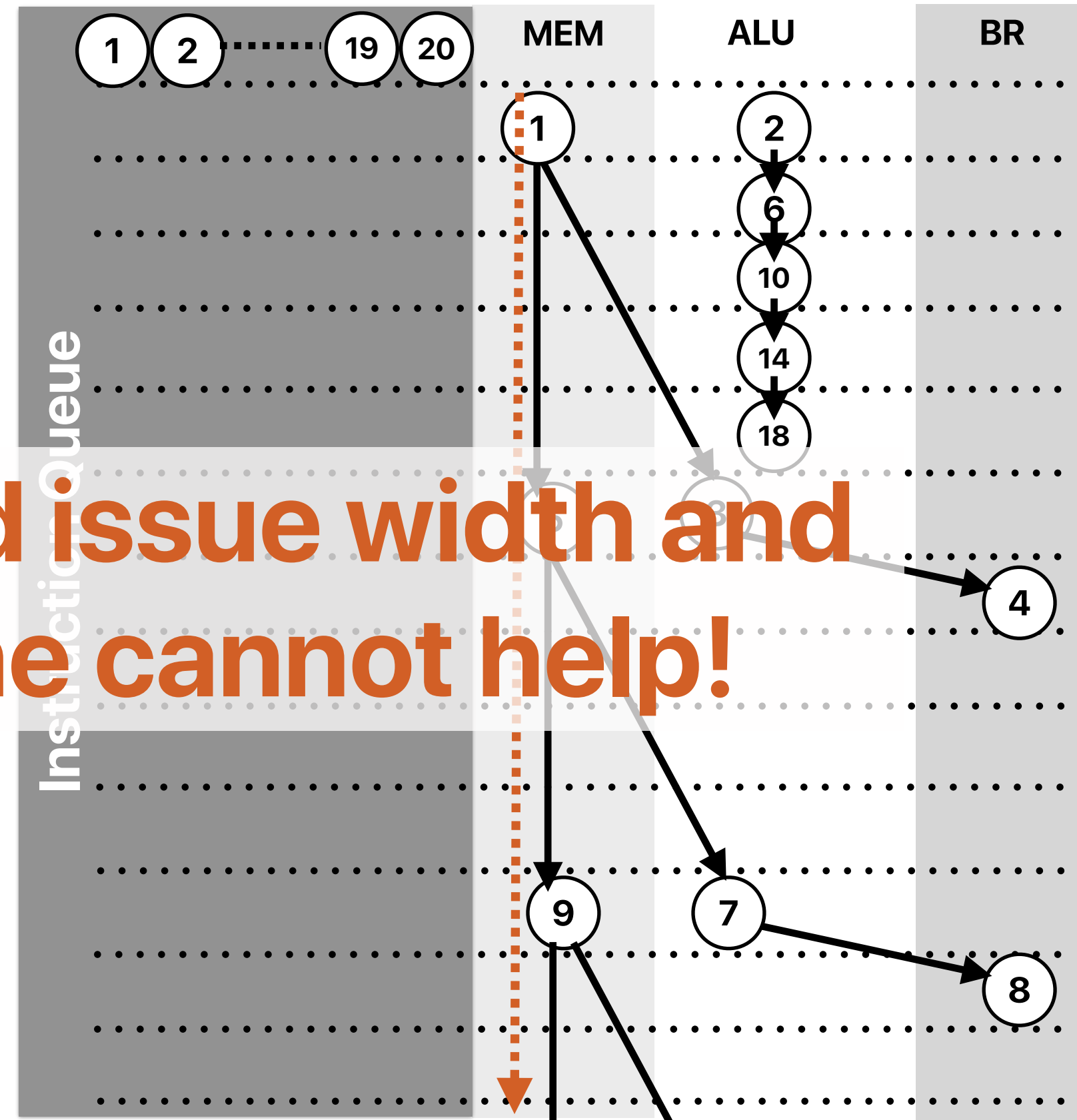
What if we have "unlimited" fetch/issue width — "linked list"

If we cannot improve the performance of executing
`movq 8(%rdi), %rdi`
we cannot improve the execution time.
That's the "critical path"!

```
do {  
    number_of_nodes++;  
    current = current->next;  
} while ( current != NULL );
```

① **.L3:** `movq 8(%rdi), %rdi`
② `addl $1, %eax`
③ `testq %rdi, %rdi`
④ `jne .L3`

Even unlimited issue width and
a perfect cache cannot help!



What about "linked list"

- For the following C code and its translation in x86, what's **average CPI**? Assume the current PC is already at instruction (1) and this linked list has thousands of nodes. This processor can fetch and issue **unlimited** number of instructions per cycle, with exactly the same register renaming hardware and pipeline as we showed previously (**5-cycle** memory access latencies, 100% hit rate).

```
do {  
    number_of_nodes++;  
    current = current->next;  
} while ( current != NULL )
```

```
① .L3:    movq    8(%rdi), %rdi  
②        addl    $1, %eax  
③        testq   %rdi, %rdi  
④        jne     .L3
```

- A. 0.5
- B. 0.75
- C. 1.0
- D. 1.25**
- E. 1.5

Linked list v.s. arrays

- We can use either a linked list or an array to store a list of data, compare the performance of the array (version A) and linked list (version B) implementations that can achieve the same outcome as below. Assume we have a processor with a reasonably good branch predictor and **unlimited** fetch/issue width and the dataset size is small enough to **fit inside the L1 cache**, please identify the correct statements.

- ① There is very little performance difference between A and B
- ② B will outperform A as A has more branch instructions
- ③ A will outperform B as A has fewer cache misses
- ✓ ④ A will outperform B as A has fewer data dependance related stalls
- ⑤ A will outperform B as A has fewer dynamic instructions

A. 0

B. 1

C. 2

D. 3

E. 4

A

```
for(i=0;i<size;i++)
{
    if(node[i].next)
        number_of_nodes++;
}
```

B

```
while(node)
{
    node = node->next;
    number_of_nodes++;
}
```

Perfectly matches our analysis!!!

size	list	IC	Cycles	CPI	CT	ET	L1_dcache_miss_rate
1024	array	514259657	106295528	0.206696	0.197243	0.020966	0.000014
1024	list	411795196	515812542	1.252595	0.196544	0.101380	0.000071

Linked-list is never an ideal option

CPI is a lot higher even with lower IC,
perfect cache and branch predictors

size	list	IC	Cycles	CPI	CT	ET	L1_dcachemiss_rate
1024	array	514259657	106295528	0.206696	0.197243	0.020966	0.000014
1024	list	411795196	515812542	1.252595	0.196544	0.101380	0.000071

size	list	IC	Cycles	CPI	CT	ET	L1_dcachemiss_rate
4096	array	205080322	41547848	0.202593	0.196833	0.008178	0.250965
4096	list	164474202	356755154	2.169065	0.196561	0.070124	0.334309
8192	array	409931842	82621771	0.201550	0.196462	0.016232	0.250467
8192	list	329389168	1048885357	3.184335	0.196532	0.206140	0.701870

\$ miss rate dominates the performance
despite lower ICs than using arrays

Takeaways: programming modern processors

- The key to efficient code is exploiting as much instruction-level parallelism (ILP) or say higher instructions per cycle (IPC) or lower cycles per instruction (CPI) as possible
 - Avoiding data dependent operations (e.g., pointer chasing)
 - Avoiding inter-iteration dependencies (e.g., linked list, trees)

Announcements

- **Assignment 4** due this upcoming **Sunday**
- **Reading Quiz 8** due **next Tuesday** before the lecture

Computer Science & Engineering

142

つづく

